

Relatório dos Slides - Aula 007

1. QUIZ DE ABERTURA: O QUE CADA LINHA IMPRIME?

Quiz 1 (Python): `print(-2 ** 2, 2 ** 3 ** 2)`

Saída: -4 512

O `**` tem precedência maior que o menos unário à sua esquerda, então `-2 ** 2` vira `-(2 ** 2)` = -4. Além disso, `**` associa da direita para a esquerda, então `2 ** 3 ** 2` vira `2 ** (3 ** 2)` = `2 ** 9` = 512.

Quiz 2 (Java): `System.out.println(1 + 2 + "3" + 4 + 5);`

Saída: 3345

A avaliação vai da esquerda para a direita. `1 + 2` é soma de inteiros e dá 3. Depois `3 + "3"` já é concatenação e dá "33". Dali em diante tudo é concatenação: "334" e depois "3345".

Quiz 3 (JavaScript): `console.log("5" - 2, "5" + 2);`

Saída: 3 52

O operador `-` só existe para números, então o JS converte "5" em 5 e dá 3. O `+` também concatena strings, e quando um dos lados é string ele concatena, resultando em "52".

Quiz 4 (C): `printf("%d\n", 3 > 2 > 1);`

Saída: 0

Em C não existe comparação encadeada. A expressão é lida como `(3 > 2) > 1`. O primeiro parêntese vale 1, e `1 > 1` é falso, que em C é 0.

Quiz 5 (Python): `x = 0 / print(x or "padrão", 3 > 2 > 1)`

Saída: padrão True

O `or` devolve um dos operandos, e não um booleano. Como 0 é falso, ele devolve "padrão". Já `3 > 2 > 1` em Python é comparação encadeada, equivalente a `(3 > 2) and (2 > 1)`, que é True.

Quiz 6 (JavaScript): `console.log(0 == "", "" == "0", 0 == "0");`

Saída: true false true

`0 == ""` dá true porque a string vazia vira o número 0. `"" == "0"` dá false porque são duas strings e são comparadas como texto, e são diferentes. `0 == "0"` dá true porque a string vira número. Isso mostra que o `==` do JS não é transitivo: `0 == ""` e `0 == "0"` são verdadeiros, mas `"" == "0"` é falso.

2. EXPRESSÕES ARITMÉTICAS: QUESTÕES DE PROJETO (Sebesta §7.2 e §7.2.1, p. 300-305)

O slide lista seis perguntas que todo projetista de linguagem precisa responder. Respondi usando Java como referência e citando Python onde muda:

- **Quais são as regras de precedência?** Em Java, os unários vêm primeiro, depois $*$ / $\%$, depois $+$ - . Python acrescenta o $**$, que fica acima do menos unário à esquerda.
- **Quais são as regras de associatividade?** Os binários aritméticos associam à esquerda. A atribuição e o $**$ (em Python) associam à direita.
- **Em que ordem os operandos são avaliados?** Em Java, sempre da esquerda para a direita. Em C, o padrão não define.
- **Há restrições a efeitos colaterais?** Java permite, mas fixa a ordem para o resultado ser previsível. Linguagens funcionais puras evitam o problema proibindo efeitos colaterais.
- **O usuário pode sobrecarregar operadores?** Em Java não, em Python sim.
- **Que mistura de tipos é permitida?** Java faz alargamento implícito (int para double, por exemplo). Go e Rust exigem conversão explícita.

O exemplo $a + b * c$ com $a=3$, $b=4$, $c=5$ mostra como a regra muda o resultado:

- Da esquerda para a direita, sem precedência: $(3 + 4) * 5 = 35$
- Da direita para a esquerda: $3 + (4 * 5) = 23$
- Com precedência ($*$ antes de $+$): $3 + 4 * 5 = 23$
- APL, sempre da direita para a esquerda: $3 \times 4 + 5 = 3 \times 9 = 27$

O Lisp dispensa a precedência, porque usa notação prefixa e os parênteses já dizem a ordem: $(+ a (* b c))$.

3. PRECEDÊNCIA E ASSOCIATIVIDADE NA PRÁTICA (§7.2.1.1 a 7.2.1.3, p. 301-305)

Tabela do slide, comparando Python, JavaScript e Java:

- $-2 ** 2$: Python dá -4. O JavaScript dá erro de sintaxe (obriga a escrever $(-2) ** 2$ ou $-(2 ** 2)$ para tirar a ambiguidade). Java não tem o operador $**$.
- $2 ** 3 ** 2$: dá 512 em Python e em JS, porque $**$ associa à direita.
- $1 + 2 + "3"$: Python dá TypeError, porque não mistura int e str. JS e Java dão "33".
- $-7 / 2$ com inteiros: Python dá -4 (usando $//$), porque a divisão inteira arredonda para baixo (piso). JS dá -3.5, porque só tem um tipo numérico. Java dá -3, porque trunca em direção ao zero.
- $-7 \% 2$: Python dá 1, JS e Java dão -1. Em Python o resto acompanha a divisão por piso e tem o sinal do divisor. Em Java e JS o resto acompanha a divisão truncada e tem o sinal do dividendo.

Associatividade (p. 303): $*$ / $\%$ + - associam à esquerda. Já $**$ (em Python, JS, Ruby e Fortran), $++$, $--$ e o menos unário associam à direita. O Visual Basic é a exceção curiosa, porque o $^$ associa à esquerda. No Excel, $=-2^2$ dá 4, porque lá o menos unário vence o $^$.

Conclusão do slide (p. 305): na dúvida, usar parênteses, porque eles deixam a intenção explícita para o compilador e para o próximo leitor.

4. ORDEM DOS OPERANDOS E EFEITOS COLATERAIS (§7.2.2, p. 307-309)

Exemplo do livro (p. 307):

```
int a = 5;
int fun1(void) {
    a = 17;    /* efeito colateral */
    return 3;
}
int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

A resposta depende da linguagem:

- Java: 8. A ordem é garantida da esquerda para a direita: lê a (5), chama fun1 (que muda a para 17, mas o 5 já foi lido) e soma 5 + 3.
- C (gcc 14): 20. O padrão não define a ordem, e o compilador pode chamar fun1 primeiro.
- Go: 20. A especificação não define a leitura de a em relação à chamada.

Lição: com efeito colateral, a ordem de avaliação muda o resultado. Sem efeito colateral existe a transparência referencial (p. 308): $f(x) + f(x)$ pode ser trocado por $2 * f(x)$ sem mudar nada, e a ordem deixa de importar.

5. OPERADORES SOBRECARGADOS (§7.3, p. 309-310 e §9.11, p. 404)

Sobrecarga é usar o mesmo símbolo com vários significados. O + soma inteiros, soma reais e concatena strings em Java. O risco (p. 309) é a legibilidade: em C++, & é E bit a bit e também "endereço de". Operadores definidos pelo usuário existem em C++, C#, Python, Rust e Kotlin (p. 404). Java não permite.

Resultados dos exemplos em Java:

- 'a' + 1 imprime 98, porque o char vira int (código 97) e soma 1.
- "" + 'a' + 1 imprime a1, porque a string na frente faz tudo virar concatenação.
- Com BigDecimal não existe +, então se usa x.add(y).multiply(tres).

No exemplo em Python, a classe Vetor define __add__ e __mul__. Por isso Vetor(1, 2) + Vetor(3, 4) vira Vetor(4, 6) e v * 3 resulta em Vetor(12, 18). Ou seja, o + chama o método especial __add__ do objeto da esquerda.

6. CONVERSÕES DE TIPOS: ALARGAMENTO, ESTREITAMENTO E COERÇÃO (§7.4, p. 310-313)

Exemplo em Java:

- `int grande = 123456789; float f = grande;` faz um alargamento implícito, mas `(int) f` imprime 123456792. O float tem só 24 bits de mantissa e não representa esse inteiro exatamente, então "alargar" aqui perdeu precisão.
- `(int) 3.99` dá 3, porque o cast trunca.
- `(byte) 200` dá -56, porque 200 passa de 127 e o valor dá a volta (200 - 256).
- `byte s = b + c` dá erro de compilação, porque `b + c` é `int` (p. 312). É preciso um cast.
- `1 / 2` dá 0 (divisão inteira) e `1 / 2.0` dá 0.5.

Tabela de linguagens (int para real implícito, e soma `int32 + int64`):

- C: sim, e faz coerção.
- Java: sim (alargamento), e faz coerção para long.
- Python: sim, e só tem um tipo int.
- Go: não, `float64(i)` é obrigatório e sem isso dá erro de compilação.
- Rust: não, `i as f64` é obrigatório, erro de compilação sem isso.
- ML e F# (p. 312): não, erro de compilação.

O livro avisa que a coerção esconde erros (p. 312): em `d = b * a`, com `a` digitado por engano no lugar de `c`, o Java converte e compila sem reclamar.

7. COERÇÃO IMPLÍCITA: JAVASCRIPT E PHP (§7.4.1, p. 311-312)

JavaScript:

- `"5" - 2` dá 3 e `"5" + 2` dá "52", como no quiz.
- `"5" * 2` dá 10, pois `*` só trabalha com números.
- `true + 1` dá 2, pois `true` vira 1.
- `[] + {}` dá "[object Object]", pois os dois viram string ("" e "[object Object]").
- `null + 1` dá 1, pois `null` vira 0.
- `undefined + 1` dá NaN, pois `undefined` vira NaN.
- `"b" + "a" + +"a" + "a"` dá "baNaNaN": o `+"a"` tenta converter "a" em número, falha e vira NaN, que depois é concatenado.
- `0 == ""` é true, `"" == "0"` é false e `0 == "0"` é true, então `o ==` não é transitivo.
- `0 === "0"` é false, porque `o ===` não converte.

PHP (type juggling):

- `"0e123" == "0e456"` dá true, porque as duas strings parecem notação científica (0 vezes 10 elevado a algo) e são convertidas para o número 0. Com `===` dá false.
- `"1" == "01"` também dá true.

- `md5("240610708")` e `md5("QNKCDZO")` começam com "0e" seguido só de dígitos.

Comparados com `==`, dão true. Por isso um login que compara hashes com `==` pode aceitar a senha errada.

Regra prática: em JavaScript e PHP, comparar sempre com `===` e `!==`.

8. EXPRESSÕES RELACIONAIS E BOOLEANAS (§7.5, p. 313-315)

Em C o booleano é inteiro (p. 315). Com $a = 3$, $b = 2$, $c = 1$:

- $a > b > c$ dá 0, porque vira $(3 > 2) > 1$, ou seja, $1 > 1$.
- $(a > b) \&\& (b > c)$ dá 1.
- $\text{if} (-1)$ é verdadeiro, porque qualquer valor diferente de zero conta como verdadeiro.

Python tem comparações encadeadas: $a > b > c$ vira $(a > b) \text{ and } (b > c)$ e dá True. Contam como falso em um if: 0, 0.0, "", [], {} e None. Contam como verdadeiro: "0", [0] e " ", pois são valores não vazios.

O que o if aceita: C, Python e JavaScript aceitam qualquer valor (0, vazio e nulo contam como falso). Java, C#, Go e Rust só aceitam booleano, e $\text{if} (n)$ com n inteiro nem compila. Isso evita vários bugs.

9. AVALIAÇÃO EM CURTO-CIRCUITO (§7.6, p. 315-317)

Na busca do livro (p. 316): $\text{while} (i < \text{lista.length} \&\& \text{lista}[i] \neq \text{chave}) i++$; o $\&\&$ para de avaliar quando o lado esquerdo é falso, então nunca acessa $\text{lista}[\text{lista.length}]$, que estouraria o vetor.

Em Java, com s nulo:

- $s \neq \text{null} \&\& s.\text{length}() > 0$ dá false, sem erro.
- $s \neq \text{null} \& s.\text{length}() > 0$ lança NullPointerException, porque o $\&$ não faz curto-circuito e avalia os dois lados.

No JavaScript, $u.\text{endereco}?.\text{cidade}$ dá undefined sem erro quando endereco é null. E há uma diferença importante entre $\|$ e $??$: com $\text{qtd} = 0$, $\text{qtd} \| 10$ dá 10 (o 0 se perdeu, porque é falso) e $\text{qtd} ?? 10$ dá 0 (o $??$ só troca quando é null ou undefined).

Cuidado com efeitos colaterais (p. 316): em $(a > b) \| (b++ / 3)$, o $b++$ só acontece quando $a \leq b$. Em Python, 0 or "padrão" devolve "padrão", pois and e or devolvem um dos operandos.

10. SENTENÇAS DE ATRIBUIÇÃO (§7.7, p. 317-321)

Em C a atribuição é uma expressão (p. 319-320), então se pode escrever:

```
sum = count = 0;
a = b + (c = d / b) - 1;
while ((ch = getchar()) != EOF) { ... }
```

O lado ruim é que $\text{if} (x = y)$ compila: atribui y a x e testa o valor atribuído, quando quase sempre o programador queria $==$.

Tabela do slide, "atribuição é expressão?":

- C, C++ e JavaScript: sim, e $\text{if} (x = y)$ compila.
- Java e C#: sim, mas o if exige booleano, então o erro é pego (exceto com booleanos).

- Python: não, exceto pelo operador morsa `:=`.
- Go: não, nem `x++` é expressão.
- Rust: a atribuição vale `()`, então `if x = 5` não compila.

Em Python existe a atribuição múltipla (`a, b = b, a`), que troca valores sem variável temporária, e a morsa (`if (n := len(dados)) > 5`), disponível desde o Python 3.8. Nas linguagens funcionais (val em ML, let em F# e Rust), o nome fica ligado a um valor que não muda (p. 321).

11. ATRIBUIÇÃO DE MODO MISTO (§7.8, p. 321-322)

Em Java só é permitido o alargamento (p. 322):

- `double d = 3`; funciona.
- `int i = 3.7`; dá erro (estreitamento).
- `int i = (int) 3.7`; dá 3, com cast explícito.
- `long l = i`; funciona.
- `i += 3.7`; compila, porque o operador composto tem um cast escondido. Com `i = 3`, o resultado é `(int)(3 + 3.7) = 6`, e o `println` imprime 6.

Em C a coerção vale em qualquer direção:

- `int i = 3.7`; vira 3 em silêncio.
- `char c = 300`; vira 44 (só dá um aviso).
- `unsigned int u = -1`; vira 4294967295.

Armadilha clássica: `double taxa = acertos / total * 100`; com inteiros dá 0.0, porque a divisão inteira acontece antes do alargamento para `double`. É exatamente o exercício 6.

12. ESTRUTURAS DE CONTROLE: QUANTAS SÃO NECESSÁRIAS? (§8.1, p. 328-330)

Existem três estruturas básicas: sequência (uma sentença após a outra), seleção (escolher um caminho) e iteração (repetir enquanto uma condição vale). Böhm e Jacopini (1966) provaram que sequência, seleção e laço lógico bastam para escrever qualquer algoritmo, então o goto não é necessário.

Se bastam três, por que existem `for`, `switch`, `do` e `foreach`? Por facilidade de escrita. O limite é a legibilidade: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

13. SELEÇÃO DE DOIS CAMINHOS: O "ELSE PENDENTE" (§8.2.1, p. 330-334)

Exemplo em C:

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
    else
        result = 1;
printf("%d\n", result);
```

Saída: 1

Apesar de a indentação sugerir que o else é do primeiro if, em Java, C, C++ e C# o else pertence ao if mais próximo que ainda não tem else, ou seja, o if (count == 0). Como sum == 0 é verdadeiro e count == 0 é falso, cai no else e result vira 1. Se o else fosse do primeiro if, nada seria executado e result continuaria -1.

A indentação é ignorada pelo compilador, exceto em Python e F#. Chaves obrigatórias (Perl, Go e Rust) eliminam o problema (p. 332).

Seletores que devolvem valor: em Rust o if é uma expressão (let y = if x > 0 { x } else { 2 * x };, como no F#, p. 334) e em Python existe a expressão condicional (y = x if x > 0 else 2 * x, p. 306).

14. SELEÇÃO MÚLTIPLA: DO SWITCH DE C AO MATCH (§8.2.2, p. 334-341)

No switch em C, sem break o controle cai para o próximo caso (p. 335). Com index = 1, ele entra no case 1, cai no case 3 (odd += 1), cai no case 2 e no case 4 (even += 1) e ainda cai no default, imprimindo "Error". Ou seja, mexe em odd, mexe em even e ainda imprime o erro.

Alternativas mais seguras:

- Java 14+: switch como expressão com ->, sem break e sem cair, exaustivo para enum.
- Rust: match exaustivo. Sem o caso _ que cobre o resto, não compila.
- Go: cada caso termina sozinho, e cair exige fallthrough explícito.
- C#: cair implicitamente é proibido (p. 336).
- Python 3.10: match com padrões e guardas.

15. LAÇOS CONTROLADOS POR CONTADOR (§8.3.1, p. 342-345)

Em C o for é um laço lógico disfarçado, porque o corpo pode alterar a variável de contagem. No exemplo, imprime 0, soma 2 (i vira 2), o i++ leva a 3, imprime 3, e assim por diante, resultando em 0 3 6 9. Também dá para omitir expressões, como em for (; i <= 100;) soma += i++;

Em Python o `for` percorre um iterável. Fazer `i += 2` dentro do corpo não muda nada, porque no início de cada volta o `i` recebe o próximo valor do `range`. Imprime 0 1 2 3 4 5 6 7 8 9. Já `range(10, 0, -3)` gera 10 7 4 1.

Questões de projeto (p. 342): qual é o tipo e o escopo da variável, se posso alterá-la no corpo, e se os parâmetros são avaliados uma vez ou a cada volta.

16. LAÇOS LÓGICOS, SAÍDAS E ITERADORES (§8.3.2 a 8.3.4, p. 345-352)

O `break` rotulado em Java (`break externo;`) sai dos dois laços de uma vez, o que evita flags auxiliares ao procurar um valor negativo numa matriz (p. 348).

No JavaScript, `for...in` percorre as chaves e `for...of` percorre os valores. Com `notas = [10, 20, 30]`, o `for...in` faz `total += i` com `i = "0", "1", "2"`. Como `i` é string, vira concatenação e o resultado é "0012". Com `for...of` o total é 60.

O caso do Zune (2008) é um laço lógico sem progresso. Se `bissexto(ano)` é verdadeiro e `dias == 366`, nenhuma das duas ramificações altera `dias` nem `ano`, então a condição `dias > 365` continua verdadeira para sempre e o laço nunca termina. O conserto é tratar o caso `dias == 366` (último dia do ano bissexto) com um `else` que saia do laço.

Lembrete: o `do-while` (pós-teste) executa ao menos uma vez, e isso é fácil de esquecer (p. 347).

17. DESVIO INCONDICIONAL: O GOTO (§8.4, p. 352-353)

No "goto fail" simplificado da Apple (2014), a primeira verificação (`verifica_hash`) passa e deixa `err = 0`. Em seguida, um `goto fail;` duplicado desvia sempre para o rótulo, sem fazer a verificação da assinatura, e a função retorna `err = 0`, que significa "certificado válido". Como não há erro, o programa não avisa que pulou a verificação.

Dijkstra (1968) escreveu "Go To Statement Considered Harmful". Hoje têm `goto` C, C++, C# e Go. Java não tem (p. 353). Gotos restritos como `break`, `continue` e `return` melhoram a legibilidade (p. 353). Com chaves obrigatórias (Go, Rust) ou com `gcc -Wall` (que avisa sobre endentação enganosa), o bug teria sido pego.

18. COMANDOS PROTEGIDOS DE DIJKSTRA (§8.5, p. 353-355)

Exemplo do livro (p. 354):

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Se `x == y`, qualquer ramo serve.

Regras:

- Todas as guardas são avaliadas.
- Se várias forem verdadeiras, a escolha é não determinística.
- Se nenhuma for verdadeira, é erro (no if) ou fim do laço (no do ... od).
- É a base do CSP de Hoare (1978) e do select de Go.

No exemplo em Go, quando os dois canais estão prontos, o select sorteia uma guarda. Em 1000 repetições deu algo como map[a:505 b:495], e esse resultado muda a cada execução, o que ilustra o não determinismo.

19. EXERCÍCIO EM DUPLAS: ENCONTRE O BUG

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

Exercício 1 (JavaScript)

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
  total += p;
console.log("Total: " + total);
```

- (a) Saída: Total: 0012
 (b) Regra: o for...in percorre as chaves (índices como string) e não os valores, e o + com string vira concatenação.
 (c) Correção: usar for...of, que percorre os valores. A saída passa a ser Total: 60.

Exercício 2 (Python)

```
def desconto(preco, d=None):
    d = d or 10 # padrão: 10%
    return preco * (100 - d) / 100
print(desconto(200, 0))
```

- (a) Saída: 180.0
 (b) Regra: o or devolve um dos operandos e o 0 conta como falso, então d = 0 or 10 vira 10. O desconto de 0% foi ignorado.
 (c) Correção: d = 10 if d is None else d, ou simplesmente declarar d=10 como valor padrão. Assim desconto(200, 0) devolve 200.0.

Exercício 3 (Java)

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
```

```
}  
System.out.println(nome);
```

- (a) Saída: inválido
- (b) Regra: sem break o controle cai para os casos seguintes. Entra no case 2, cai no case 3 e no default, e o último valor atribuído é "inválido".
- (c) Correção: colocar break ao final de cada caso, ou usar o switch com -> do Java 14+.

Exercício 4 (C)

```
int saldo = 100, saque = 50;  
if (saque > saldo);  
    printf("saldo insuficiente\n");  
printf("fim\n");
```

- (a) Saída: "saldo insuficiente" e depois "fim" (as duas linhas, mesmo com o saque menor que o saldo).
- (b) Regra: o ; logo depois do if é um comando vazio. O if controla só esse comando vazio, e o printf seguinte é sempre executado.
- (c) Correção: remover o ponto e vírgula e usar chaves: if (saque > saldo) { printf("saldo insuficiente\n"); }. A saída passa a ser só "fim".

Exercício 5 (Go)

```
x := 10  
if x > 5 {  
    x := x * 2  
    fmt.Println("dentro:", x)  
}  
fmt.Println("fora:", x)
```

- (a) Saída: "dentro: 20" e "fora: 10"
- (b) Regra: o := declara uma nova variável. Dentro do if nasce um x novo que esconde (shadowing) o x de fora, e quando o bloco termina esse x some.
- (c) Correção: se a intenção era alterar o x de fora, usar x = x * 2 (atribuição, sem os dois pontos). Aí a saída é "dentro: 20" e "fora: 20".

Exercício 6 (Java)

```
int acertos = 7, total = 10;  
double taxa = acertos / total * 100;  
System.out.println(taxa + "%");
```

- (a) Saída: 0.0%
- (b) Regra: 7 / 10 é divisão inteira e dá 0. Depois 0 * 100 = 0, e só então o resultado é alargado para double.
- (c) Correção: forçar a conta em double antes da divisão, como (double) acertos / total * 100, ou acertos * 100.0 / total. A saída passa a ser 70.0%.

20. SÍNTESE: ARMADILHAS E COMO AS LINGUAGENS AJUDAM

- Precedência: $-2^{**}2$ e $1 + 2 + "3"$. Ajuda usar parênteses, e o JS obriga a escrever $(-2)^{**}2$.
- Operandos: $a + f()$ com efeito colateral (C, Go). O Java fixa a ordem, e funções puras evitam o problema.
- Conversões: coerção silenciosa (C, JS, PHP). Go e Rust exigem conversão explícita, e em JS e PHP se usa `===`.
- Booleanos: 0, "" e [] são falsos, e $a > b > c$ em C. Java, Go e Rust só aceitam booleano no if, e o JS tem o `??`.
- Atribuição: `if (x = y)`. Em Python, Go e Rust a atribuição não é expressão.
- Seleção: switch sem break e else pendente. Ajudam o `->`, o match exaustivo e as chaves obrigatórias.
- Laços: `for...in` no JS e laço sem progresso. Ajudam o `for...of`, os iteradores e o break rotulado.
- Desvio: goto sem restrição. Ajudam break, continue e return, que são gotos restritos.

21. ATIVIDADES COMPLEMENTARES (AUTOPESQUISA)

Expressões em Ruby e em Lisp (§7.2.1.4 e 7.2.1.5, p. 305-306): em Ruby, praticamente tudo é expressão (até o if devolve valor) e os operadores são métodos, então $a + b$ equivale a $a.+(b)$. Em Lisp a notação prefixa elimina a necessidade de regras de precedência e associatividade.

Expressões condicionais (§7.2.1.6, p. 306): é o operador ternário (condição ? $a : b$), presente em C, Java e JS, equivalente à forma `x if cond else y` do Python.

Erros em expressões (§7.4.3, p. 313): overflow, underflow e divisão por zero. Em Java, o overflow de inteiros dá a volta em silêncio, e a divisão inteira por zero lança `ArithmeticException`.

Implementação da seleção múltipla (§8.2.2.3, p. 337-339): o compilador pode usar uma tabela de desvio, em que o valor do seletor serve de índice para o endereço do caso. É rápido (tempo constante) quando os casos são densos. Para casos esparsos, pode usar busca ou comparações sequenciais.

Laços em linguagens funcionais (§8.3.1.4, p. 345): sem variável mutável, a contagem é feita por recursão, passando o contador como parâmetro.

Iteradores e blocos de Ruby (§8.3.4, p. 349-352): métodos como `each` e `times` recebem um bloco, e o iterador chama o bloco a cada elemento (via `yield`).

Leitura de Dijkstra (1968): o argumento central é que o goto dificulta acompanhar o estado do programa e o fluxo de execução, e que a qualidade do código piora com o uso dele.

22. ENTREGA SUGERIDA: SELEÇÃO MÚLTIPLA EM QUATRO LINGUAGENS

O mesmo exemplo (descrever um número de 1 a 3) em C, Java, Go e Rust.

C (precisa de break, senão cai):

```
switch (n) {  
    case 1: printf("um\n"); break;  
    case 2: printf("dois\n"); break;  
    case 3: printf("tres\n"); break;  
    default: printf("outro\n");  
}
```

Java 14+ (switch como expressão, sem cair):

```
String nome = switch (n) {  
    case 1 -> "um";  
    case 2 -> "dois";  
    case 3 -> "tres";  
    default -> "outro";  
};
```

Go (cada caso termina sozinho):

```
switch n {  
case 1:  
    fmt.Println("um")  
case 2:  
    fmt.Println("dois")  
case 3:  
    fmt.Println("tres")  
default:  
    fmt.Println("outro")  
}
```

Rust (match exaustivo):

```
let nome = match n {  
    1 => "um",  
    2 => "dois",  
    3 => "tres",  
    _ => "outro",  
};
```

Comparação:

- Cair entre casos: só o C cai por padrão. Java 14+ com -> e Rust não caem, e o Go só cai com fallthrough.

- Exaustividade: o Rust obriga a cobrir todos os casos (sem o `_` não compila), e o Java com `->` exige cobrir todos os casos quando o `switch` é expressão. C e Go não exigem.
- Legibilidade: Rust e Java com `->` são mais curtos e sem risco de esquecer o `break`. O C é o que mais depende da atenção do programador.

23. CONCLUSÃO

A aula mostrou que quase todo bug dos exemplos nasce de uma regra da linguagem que o programador não tinha em mente: precedência, coerção silenciosa, ordem de avaliação, valores falsos, atribuição dentro de condição, `switch` sem `break`, laço sem progresso ou `goto`. As linguagens modernas ajudam a evitar esses erros exigindo booleano no `if`, chaves obrigatórias, `match` exaustivo e conversões explícitas, mas isso não dispensa o cuidado de quem escreve. Como engenheiro de software, a prática que mais fica é usar parênteses na dúvida, comparar com `===` em JS e PHP, usar chaves sempre, e ler com atenção o que a linguagem realmente faz, e não o que a indentação sugere.